

TestMachine_OOP – ST-OOP Enumeration

Reproduction Case

Summary

This package contains two Automation Studio projects that were created to investigate a potential limitation or defect related to the use of ENUM data types within ST-OOP Function Blocks.

The projects are intentionally kept small and focused on a single topic to simplify analysis and reproduction.

The objective is to determine whether the observed behavior is:

- expected and documented,
 - a known limitation of the current ST-OOP implementation,
 - a configuration issue,
 - or a defect within the compiler and/or code generation toolchain.
-

Development Environment

Software

- Automation Studio 6.5.2.12
- Automation Studio Code 6.5.2

Target

- Standard PC Runtime
- Automation Runtime Simulation

Operating System

- Windows 10 Professional
-

Included Projects

The package contains two projects:

TestMachine_OOP_Working

Reference project.

This version compiles successfully.

The internal state variables of the state machine Function Block are implemented using the data type:

```
INT
```

TestMachine_OOP_Failing

Reproduction project.

This version fails to compile.

The internal state variables of the state machine Function Block are implemented using the ENUM type:

```
E_MachineState
```

Project Description

Both projects implement a simple ST-OOP state machine:

```
FB_StateMachine
```

Multiple instances of this Function Block are declared:

```
MachineSM  
ModuleA_SM  
ModuleB_SM  
ModuleC_SM  
ModuleD_SM  
ModuleE_SM  
ModuleF_SM  
ModuleG_SM
```

The application itself is intentionally simple and only serves as a vehicle for reproducing the observed behavior.

Observed Behaviour

Working Version

Project:

```
TestMachine_OOP_Working
```

Build result:

```
Build successful
```

Failing Version

Project:

```
TestMachine_OOP_Failing
```

Build result:

```
Error 5885:  
The initialisation of IEC OOP variables is not supported
```

The error is reported in:

```
Logical\PrgMain\Variables.var
```

on a declaration similar to:

```
ModuleA_SM : FB_StateMachine;
```

Files Modified Between Both Projects

Only a small number of files were intentionally modified.

File

```
Logical\PrgMain\Types.typ
```

Working Version

No ENUM state definition used.

Failing Version

Contains:

```
TYPE

  E_MachineState :
  (
    STATE_INIT := 0,
    STATE_HOMING := 5,
    STATE_IDLE := 10,
    STATE_STARTING := 20,
    STATE_RUNNING := 30,
    STATE_STOPPING := 40,
    STATE_ERROR := 100
  );

END_TYPE
```

File

```
Logical\PrgMain\FB_StateMachine.fub
```

Working Version

```
VAR
  State : INT;
END_VAR
```

```
VAR_OUTPUT
    StateOut : INT;
END_VAR
```

Failing Version

```
VAR
    State : E_MachineState;
END_VAR

VAR_OUTPUT
    StateOut : E_MachineState;
END_VAR
```

File

```
Logical\PrgMain\FB_StateMachine.stpp
```

Working Version

State transitions use integer values:

```
State := 0;
State := 5;
State := 10;
State := 20;
State := 30;
State := 40;
State := 100;
```

Failing Version

State transitions use ENUM values:

```
State := STATE_INIT;
State := STATE_HOMING;
State := STATE_IDLE;
State := STATE_STARTING;
State := STATE_RUNNING;
```

```
State := STATE_STOPPING;  
State := STATE_ERROR;
```

Reproduction Procedure

Step 1

Open project:

TestMachine_OOP_Working

Build configuration:

Config1

Expected result:

Build successful

Step 2

Open project:

TestMachine_OOP_Failing

Build configuration:

Config1

Observed result:

Error 5885:
The initialisation of IEC OOP variables is not supported

Additional Notes

The state machine implementation contains simple homing, starting and stopping delays.

To keep the project focused on the ST-OOP investigation, these delays are implemented using cyclic counter increments (scan-count based timing) rather than IEC timer function blocks.

Example:

```
StepCounter := StepCounter + 1;
```

The timing implementation is not considered part of the issue being reported and was only used to simulate state transitions during testing.

Questions

1. Are ENUM variables inside ST-OOP Function Blocks officially supported?
 2. Are multiple instances of an ST-OOP Function Block containing ENUM members officially supported?
 3. Is the observed behavior a known limitation of the current ST-OOP implementation?
 4. Is additional syntax, configuration or project setup required when using ENUM members inside ST-OOP Function Blocks?
 5. Is Error 5885 expected behavior in this situation, or should it be considered a compiler/toolchain defect?
-

Goal of this Report

The intention of this package is to provide a minimal and reproducible example that allows the observed behavior to be verified quickly and independently.

Any clarification regarding intended usage, current limitations, or potential defects would be greatly appreciated.